Deep Learning review

Manuel

February 25, 2026

1 MLP 2

2 Optimization: Gradient Descent 3

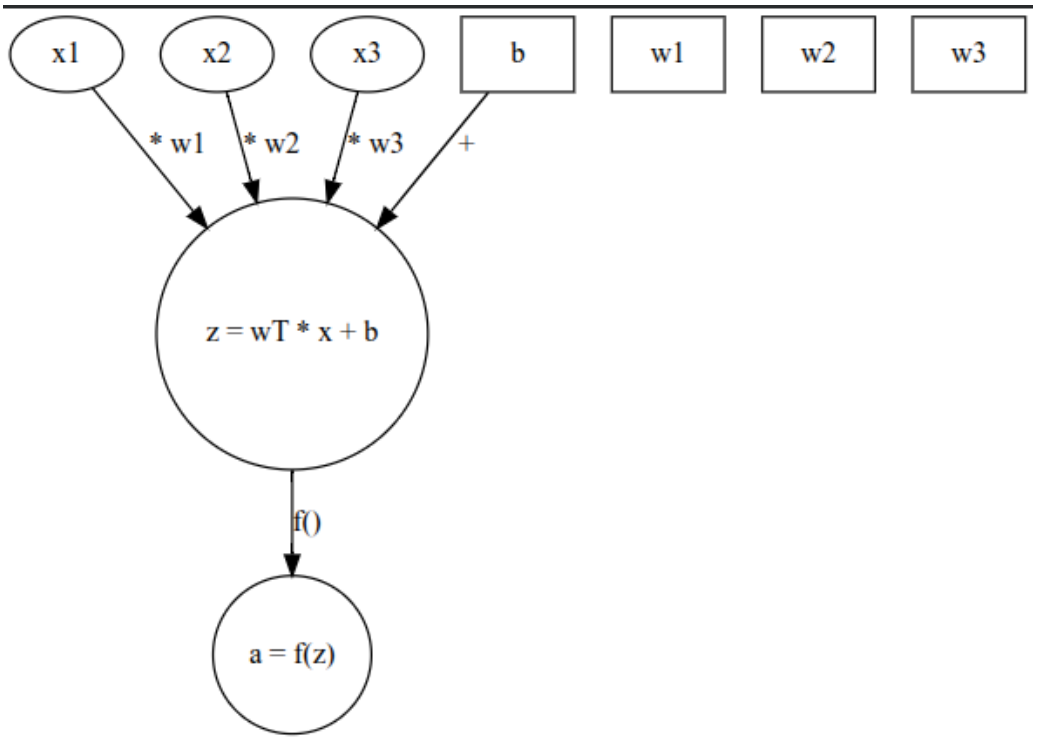
3 Backpropagation 5

4 Recurrent Neural Networks (RNN) 8

5 Long Short-Term Memory (LSTM) 13

6 Transformers 16

MLP



Gradient Descent: The Optimization Engine

The Update Rule

For a set of parameters θ, the update at step t is:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_\theta J(\theta_t)$$

Where:

- η: **Learning rate** (step size).
- $\nabla_\theta J(\theta_t)$: **Gradient vector** (direction of steepest ascent).

Gradient Descent: Pros and Cons

Pros	Cons
Conceptually simple and theoretically grounded in calculus.	Risk of getting stuck in **local minima** or **saddle points**.
Scalable to millions of parameters (via SGD/Adam).	Highly sensitive to the choice of the learning rate (η).
Guaranteed convergence for convex loss functions.	Computationally expensive if the full dataset is used (Batch GD).

Transition to Backpropagation: In deep networks, calculating $\nabla_\theta J$ directly is intractable. We need an efficient way to apply the **Chain Rule** from the output back to the input.

BACKPROPAGATION

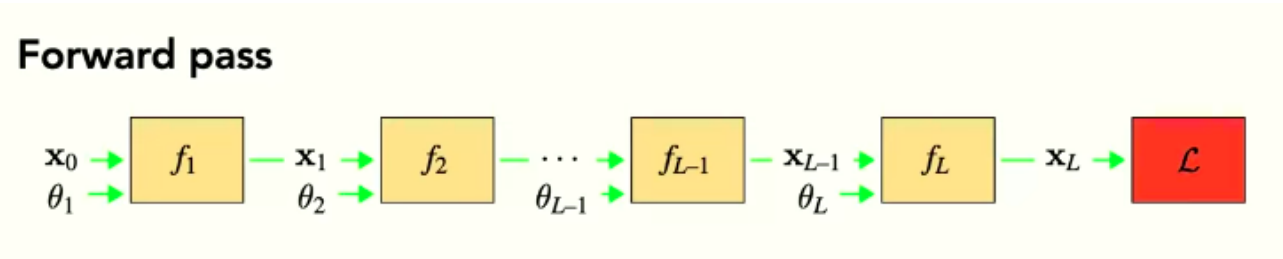


Figure. Caption

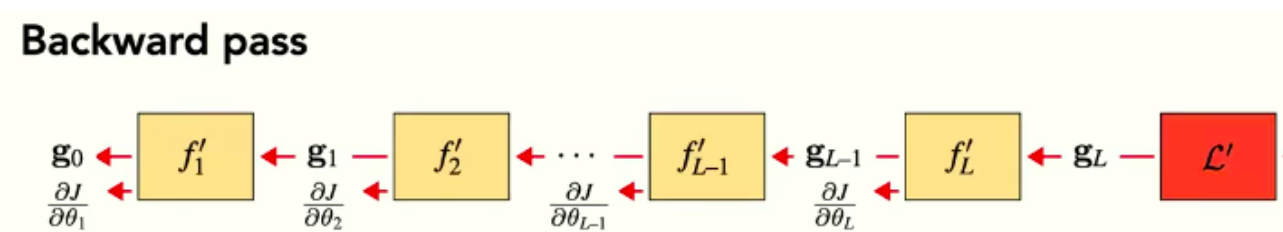


Figure. Caption

Backpropagation

$$
\begin{aligned}
\partial J/\partial\theta_1 &= \partial J/\partial x_L \partial x_L/\partial x_{L-1} \cdots \partial x_2/\partial x_1 \partial x_1/\partial\theta_1 \\
\partial J/\partial\theta_2 &= \partial J/\partial x_L \partial x_L/\partial x_{L-1} \cdots \partial x_3/\partial x_2 \partial x_2/\partial\theta_2 \\
&\vdots
\end{aligned}
$$

Updating the weights to minimize the cost:

$$
\theta^{i+1} = \theta^i - \eta \frac{\partial J}{\partial \theta}^T \tag{1}
$$

BACKPROPAGATION ALGORITHM

Input: parameter vector $\theta = \{\theta_l\}_{l=1}^{L}, f_1, \ldots, f_L, f_1', \ldots, f_L'$, training datapoint $\{\mathbf{x}_0, \mathbf{y}\}$, Loss function $\mathcal{L} : \mathbb{R}^N \to \mathbb{R}$

Output: gradient direction $\frac{\partial J}{\partial \theta} = \left\{ \frac{\partial J}{\partial \theta_l} \right\}_{l=1}^{L}$

Forward pass:

for $l = 1, \ldots, L$ **do**

  $\mathbf{x}_l = f_l(\mathbf{x}_{l-1}, \theta_l)$

Backward pass:

$\mathbf{g}_L = \mathcal{L}'(\mathbf{x}_L, y)$

for $l = L, \ldots, 1$ **do**

  $\mathbf{L}_l = f_l'(\mathbf{x}_{l-1}, \theta_l)$

  $\mathbf{g}_{l-1} = \mathbf{g}_l \mathbf{L}_l^{\mathbf{x}}$

  $\frac{\partial J}{\partial \theta_l} = \mathbf{g}_l \mathbf{L}_l^{\theta}$

Figure. Caption

Recurrent Neural Networks (RNN)

- RNNs are designed to handle sequential data (text, audio, time series).
- Hidden State (h_t): The network maintains an internal "memory" that is updated at each time step.

Mathematical Model

At each time step t, the hidden state h_t and output y_t are calculated as:

$$h_t = \phi(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$y_t = \mathrm{f}(W_{hy}h_t + b_y)$$

RNN

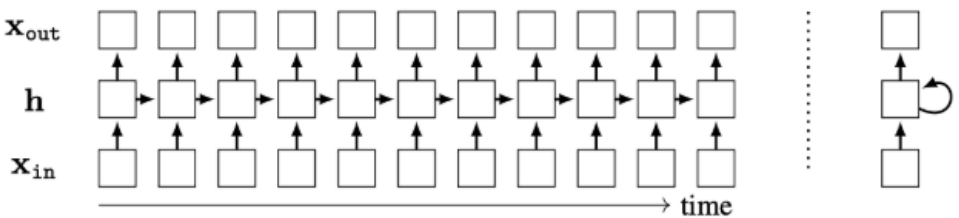


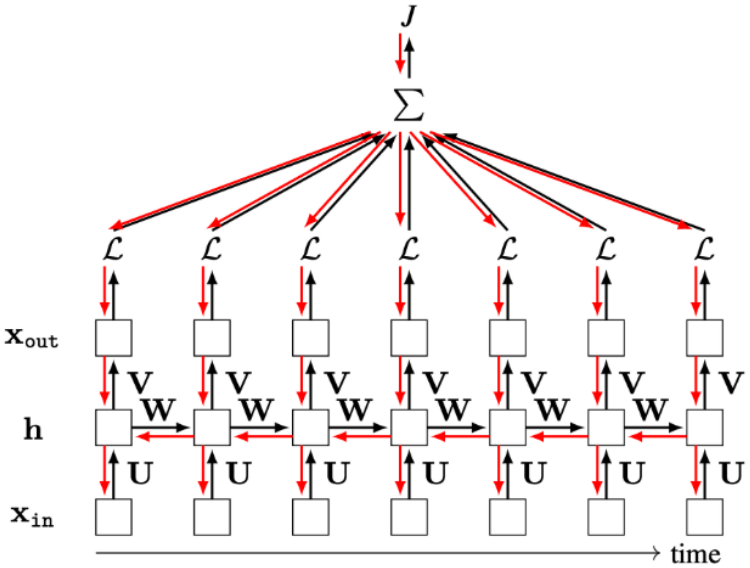
Figure. Caption

RNN

Pros	Cons
Variable Length: Can process sequences of any size.	**Sequential Bottleneck:** Cannot process words in parallel (slow training).
Historical Context: Uses previous information to influence current prediction.	**Short-term Memory:** In practice, they struggle to link information that is far apart.
Parameter Efficiency: Uses the same weights at every step.	**Numerical Instability:** Prone to Vanishing and Exploding gradients.

The Gradient Problem in RNNs

Backpropagation Through Time (BPTT): Gradients are calculated by unfolding the RNN.



Recurrent Neural Networks (RNN)

The Gradient Product

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}} = \prod_{k=t+1}^{T} W_{hh}^T \mathrm{diag}(\sigma'(\ldots))$$

LSTM

- LSTMs introduce the Constant Error Carousel (CEC), implemented via a "Cell State" (c_t).
- Instead of simple multiplication, LSTMs use gates to control information flow

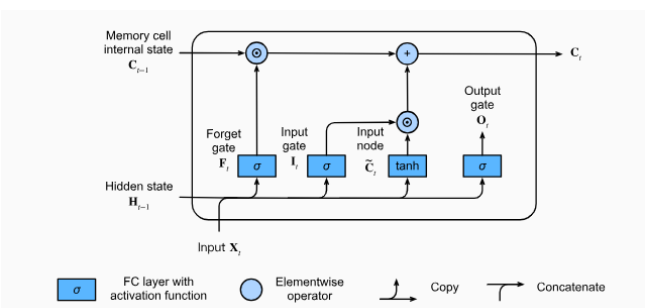


Figure. Caption

Long Short-Term Memory (LSTM)

The Gates

- input node: $\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t\mathbf{W}_{\mathrm{xc}} + \mathbf{H}_{t-1}\mathbf{W}_{\mathrm{hc}} + \mathbf{b}_{\mathrm{c}),}$
- memory cell internal state: $\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t.$
- hidden state: $\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t).$

LSTM: Pros and Cons

Pros

- The additive update of the cell state prevents vanishing gradients.
- Handled complex NLP tasks like translation effectively.

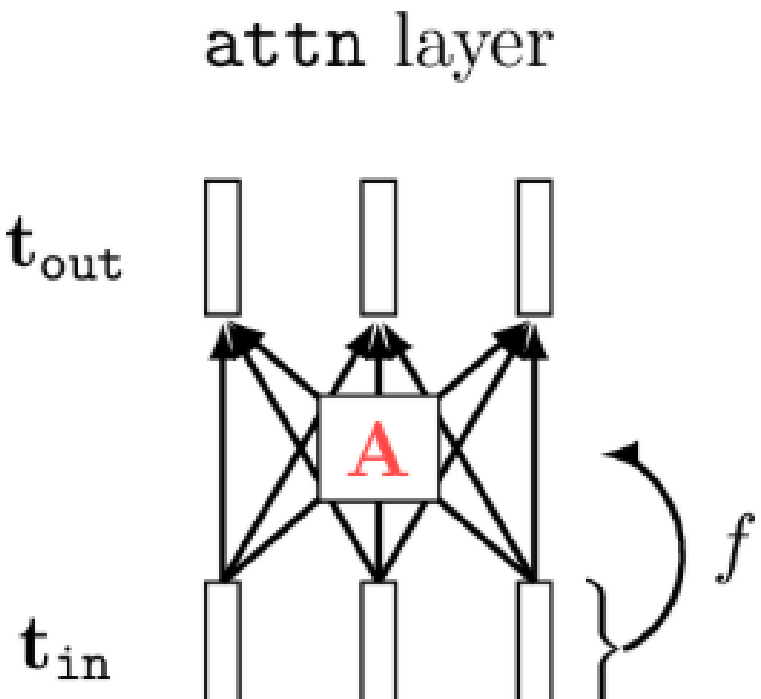
COns

- Like RNNs, they cannot process words in parallel (GPU underutilization)
- 4x more parameters than a vanilla RNN cell
- Still assumes that "closer" words are always more relevant.

Transformers

- The Problem with RNNs/LSTMs: Sequential processing is a bottleneck. We cannot compute word t until $t - 1$ is finished.
- The Solution: Self-Attention. This mechanism allows the model to look at all words in a sequence simultaneously and weigh their relevance to each other.

SELF ATENTION



SELF ATENTION

$$\mathbf{Q}_{\text{in}} = \begin{bmatrix} \mathbf{q}_1^\top \\ \vdots \\ \mathbf{q}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_q \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_q \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_q^\top \qquad \triangleleft \qquad \text{query matrix}$$

$$\mathbf{K}_{\text{in}} = \begin{bmatrix} \mathbf{k}_1^\top \\ \vdots \\ \mathbf{k}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_k \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_k \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_k^\top \qquad \triangleleft \qquad \text{key matrix}$$

$$\mathbf{V}_{\text{in}} = \begin{bmatrix} \mathbf{v}_1^\top \\ \vdots \\ \mathbf{v}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_v \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_v \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{\text{in}} \mathbf{W}_v^\top \qquad \triangleleft \qquad \text{value matrix}$$

Figure. Caption

The Scaling Dot-Product Attention

$$\text{Attention}(Q, K, V) = f(T_{\text{in}})\text{softmax}\left(\frac{Q_{\text{in}}K_{\text{in}}^T}{\sqrt{m}}\right)V$$

- **Queries (Q):** What I am looking for.
- **Keys (K):** What I contain.
- **Values (V):** The information I provide.

Transformers: Pros and Cons

Pros	Cons
Parallelization: Processes the entire sequence at once (efficient GPU training).	**Quadratic Complexity:** Self-attention cost is $O(n^2)$ relative to sequence length.
Global Context: Can link any two words regardless of their distance.	**No Inherent Order:** Needs "Positional Encodings" to know the word order.
Scalability: Foundation for Large Language Models (LLMs) like GPT and BERT.	**Data Hungry:** Requires massive amounts of data to learn effectively.

References I